

# 0.70. Plus fluide : la lettre au pixel près

`sprite(0, px, py, ALPHABET[0])` : la lettre devient un lutin, qui avance d'UN pixel par image au lieu de sauter d'une case. Le mouvement devient fluide.

## 1. Ce qu'il faut comprendre

**Pourquoi la lettre saute** : jusqu'ici, elle était **posée sur la grille** avec `poser`. Une case fait **8 × 8 pixels** : chaque pas la faisait donc sauter de 8 pixels d'un coup, environ 7 fois par seconde. L'œil voit des sauts.

**Ce qui est nouveau ici** : un **lutin**, avec `sprite(numero, x, y, tuile)`. Un lutin n'est **pas** sur la grille : il flotte au-dessus du décor, et se place **au pixel près**. On le fait avancer d'un **seul pixel par image**, 60 fois par seconde : le mouvement devient **fluide**.

- `numero` : le numéro du lutin, de 0 à 39 (la console en a 40). Ici, `0`, le premier.
- `px`, `py` : sa place **en pixels**, et plus en cases. L'écran fait **160 pixels** de large et **144** de haut. Le milieu, pour une lettre de 8 pixels :  $(160 - 8) \div 2 = 76$ , et  $(144 - 8) \div 2 = 68$ .
- `tuile` : ce qu'il montre, `ALPHABET[0]`, le A.

**Plus besoin d'effacer ni d'attente** : un lutin se **déplace**, il ne se réécrit pas. On lui donne sa nouvelle place à chaque image, et la console le dessine là, sans rien laisser derrière. Et comme un pixel est petit, on peut avancer à chaque image, sans attendre.

**Les bords** : `px` va de 0 à **152** ( $160 - 8$ , pour que la lettre reste entière), `py` de 0 à **136** ( $144 - 8$ ).

### Les fonctions de cette leçon — et le tuto de chacune

- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **340 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          55 o ← la console, à l'allumage — la mise en route de la console
main()             113 o ← démarrage — ta fonction
VBlank              8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage       28 o ← image() — attend l'image suivante
AttendreVBlank      16 o ← démarrage — attend que l'écran soit libre
RangerLutins        11 o ← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert  15 o ← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles        31 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
LireManette         37 o ← bouton() — lit les huit boutons
Tuiles              16 o ← CopierTuiles — 1 lettre sur 43
DonneesTransfert    10 o ← InstallerTransfert — la petite routine qui copie les lutins
```

Lettres gravées : A

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, EffacerCarte, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

```
sprite() × 1    23 o sur place, dans main()
bouton() × 4    20 o sur place, dans main() + LireManette (37 o)
image() × 1     6 o sur place, dans main() + AttendreImage (28 o)
```

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// px, py : voir l'onglet variables.h

#include <bouton>      // lit un bouton de la manette
#include <sprite>      // place un lutin de 8 × 8 au pixel près
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include "variables.h"
```

## 2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();                                // 60 fois par seconde

        // UN pixel par image, tant que la flèche est tenue : 60 pixels par
        // seconde, sans à-coups. (Avec poser, c'était 8 pixels d'un coup.)
        if (bouton(DROITE) && px < 152) px++;    // 152 = 160 - 8 : le bord droit
        if (bouton(GAUCHE) && px > 0)  px--;
        if (bouton(BAS) && py < 136)  py++;    // 136 = 144 - 8 : le bord du bas
        if (bouton(HAUT) && py > 0)    py--;

        // Le lutin, morceau par morceau :
        //
        //  sprite(0, px, py, ALPHABET[0]);
        //      | | | |
        //      | | | +-- tuile : ce qu'il montre, la lettre A
        //      | +---+----- px, py : sa place EN PIXELS (et non en cases)
        //      +----- numero : le lutin n° 0 (la console en a 40, de 0 à 39)
        //
        // Pas d'effacer : un lutin se DÉPLACE, il ne laisse rien derrière lui.
        sprite(0, px, py, ALPHABET[0]);
    }
}
```

## 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



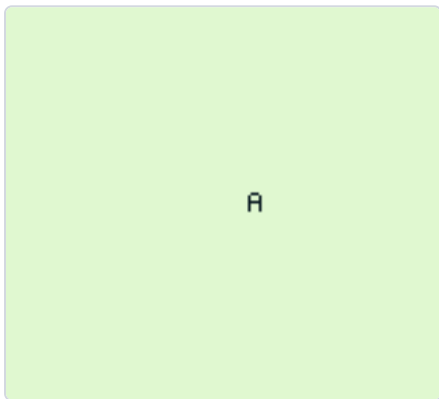
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

## 4. Ce que les touches changent

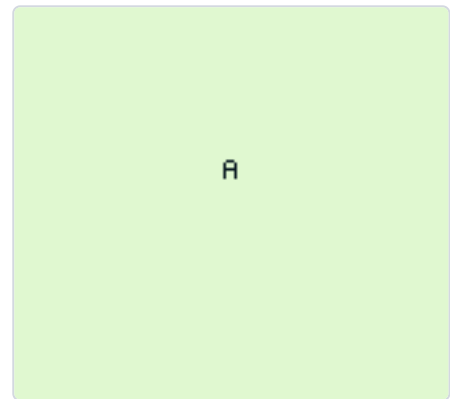
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



**DROITE** — l'écran a changé



**GAUCHE** — l'écran a changé



**HAUT** — l'écran a changé



**BAS** — l'écran a changé

## 5. Ce qu'on doit voir

Un A au milieu de l'écran, qui glisse doucement avec les flèches, pixel par pixel, sans sauter de case en case.

## 6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le lutin A part du milieu (76, 68) (76, 68)

✓ 30 images à droite : 30 pixels, un par image (30)

✓ 20 images en bas : 20 pixels (20)

✓ il s'arrête au bord gauche (0) ( $x = 0$ )

---

Ensuite : **0.70.1. Plus fluide : la lettre au pixel près — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++